

FIT/CAR AGV Prototype Software Blueprint

Brian L. Stuart

July 5, 2006

This report describes the initial development software for the FIT/CAR AGV project. It is meant to be a living document giving the conventions and techniques that we use in development.

1 General Architecture

The AGV system is composed of a number of components, each running one or more threads, built around a blackboard. Most threads fit into one of three roles:

Effector: Effectors take data from the blackboard and transform it into actions in the real world. These threads control vehicle hardware. In the current prototype collection of components, there is a component for controlling the P3 robot which takes blackboard postings from an arbitrator and turns them into commands for the P3 robot.

Sensor: Threads in the sensor role primarily post to the blackboard. In most cases, they post data taken from some type of input device such as a camera or a SONAR array.

Processor: The third role for component threads is the processor. These threads take data from the blackboard, transform it into another form and post results back. In the current set of component, the navigator and arbitrator are two examples of processors. The navigator takes data from the localization component and issues motion requests in the form of velocities and turning radii. The arbitrator takes the postings of both the navigator component and the obstacle handling component and selects the proper course of action from them. It then posts back to the blackboard that action for the robot control component to use in commanding the robot.

The set of prototype components is illustrated in Figure 1.

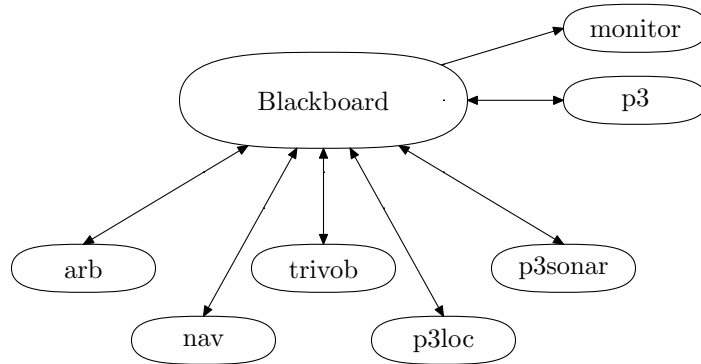


Figure 1: Current Component Set

2 Blackboard

The blackboard is the central organizing element of this design. All component communication takes place through it. It has intentionally been designed to be simple, even primitive. Each data item on the blackboard has a unique label and a timestamp recording the time it was posted. Data items are never removed from the blackboard and new postings with an existing label replace the previous data with that label. Components communicate with the blackboard through four calls:

- `int bb_post( long label, int n, void *data ) ;`

This is the call components use to publish information to the blackboard. The first argument specifies the unique label for this data. The second gives the size of the data element in bytes, and the third argument is a pointer to the data itself. The blackboard only stores the pointer to the data and does not make a copy of the data before storing it in the blackboard. Consequently, a component that publishes data to the blackboard must be careful not to free nor to overwrite the data once it's been posted.

- `void *bb_read( long label, int *n, struct timeval *stamp ) ;`
`void *bb_copy( long label, int *n, struct timeval *stamp ) ;`

These two calls are the means by which a client component can retrieve data from the blackboard. The first argument specifies the label for the desired data. If non-NULL, the second and third arguments point to locations where the blackboard can store the size of the item and the timestamp respectively. If the data item exists on the blackboard, these calls return a pointer to the data. The `bb_read()` call returns the same pointer as was passed in the `bb_post()` call. In other words, the poster of the data and the consumer of the data share the memory the data occupies. The `bb_copy()` call makes a copy of the data and returns a

pointer to the copy. In this case, the component making the `bb_copy()` call is responsible for freeing the memory occupied by the copy. For most data items, clients should use `bb_copy()` rather than `bb_read()` in order to avoid race conditions. However, for very large data, such as images, `bb_read()` may be preferable. In those cases, the data item should include a mutex lock to protect critical sections. If the requested data item is not found on the blackboard, these calls return NULL.

- `long bb_wait( long *labels, struct timeval **stamps, int nlabel ) ;`

Most components repeatedly read one or more data items from the blackboard. We don't want them consuming CPU time between new postings of the data that interests them. The `bb_wait()` call lets components subscribe to one or more data items and blocks until a new one of them appears. The first argument points to a list of labels, any one of which can wake the component up when it gets posted. The second argument optionally points to a list of timestamp pointers. If non-NULL, a timestamp indicates that only data items newer than that time are eligible to wake the component up. Finally, the third argument gives the number of labels and timestamps in the lists. The function returns the label which caused it to wake up.

3 Components

The following sub-sections describe the current set of components. They are listed in alphabetical order by the component name in the source code.

3.1 arb

This is the arbitrator component. It's primary function is to resolve potentially conflicting instructions from the navigation and the obstacle handling components. The current version subscribes to `OB_STATE` and `NAV_STATE` postings from the two other components. If the obstacle handler says we need to stop, we post a message saying so. Otherwise, we pass on whatever navigational request comes from the navigator.

The arbitrator posts only one data structure, the `ARB_CMD` to the blackboard. This structure is defined as:

```
struct arb_cmd {
    int op ;
    long vel ;
    long radius ;
} ;
```

The `op` field can take on one of two values, `ARB_STOP` and `ARB_MOVE`. If it equals `ARB_STOP`, the robot should stop moving and ignore the values of `vel`

and **radius**. If it equals **ARB_MOVE**, the robot should move at **vel** mm/S and with a turning radius of **radius** mm. A **radius** of 0 indicates no turn, a positive **radius** indicates a turn to the left and a negative **radius** indicates a turn to the right.

3.2 monitor

The monitor component is sort of a null effector. It reads the current status from the blackboard and displays it on the screen but controls nothing in the real world. Specifically, it reads the **P3_STATE** posting and prints out the values of the motor status, the (x, y) position, the heading angle and the left and right wheel velocities. We anticipate that this component will later be expanded into one that a network monitor/control program can connect to and have a more complete view into the system's operation.

3.3 nav

The current version of the simplistic navigator just moves the robot in a square pattern once. The next development step is to change it to follow a set of waypoints posted to the blackboard.

Whenever the navigator determines that a change in the vehicle's motion is needed, it posts a **NAV_CMD** message to the blackboard. The accompanying data is defined by:

```
struct nav_cmd {
    int op ;
    long vel ;
    long radius ;
} ;
```

where **op** is either **NAV_STOP** or **NAV_MOVE** indicating whether the vehicle should stop or move. When moving, **vel** specifies the vehicle velocity in mm/S and **radius** specifies the turning radius in mm. A positive **radius** indicates a left-hand turn and a negative **radius** a right-hand turn.

3.4 p3

The p3 component is the most complex of the current components. Its primary task is reading instructions from the arbitrator and sending the P3 robot commands. At initialization, it splits itself into three threads. One thread is just a heartbeat that sends the the robot a packet once a second to reset its watchdog timer. One thread is a sensor thread that reads periodic status packets from the robot and posts status updates to the blackboard. The third thread reads **ARB_CMD** postings from the blackboard and translates them into the necessary command sequences for the P3.

Only the sensor thread posts to the blackboard. It posts a **P3_STATE** message with data defined as follows:

```

struct p3_state {
    unsigned char mstate ;
    short xpos, ypos, angle ;
    short lvel, rvel ;
    unsigned char batt ;
    int n_sonar ;
    unsigned short srange[P3_MAX_SONAR] ;
} ;

```

The value of `mstate` indicates whether the motors are currently operating or not. The current position and orientation are given by `xpos`, `ypos` and `angle`. Current velocities for both wheels are given by `lvel` and `rvel`. The value of `batt` gives the current battery charge. Finally, `n_sonar` and `srange` provide the raw sonar data from the robot. Most components should not use these data directly as the details of units and format are dependent on the P3 robot. Other components present these data in a more generic form.

3.5 p3loc

This component is a trivial localization component based on the encoder odometry from the P3 robot. It monitors the `P3_STATE` postings and posts generic localization state information for other components.

The `p3loc` component posts `LOC_STATE` structures defined by:

```

struct loc_state {
    int xpos ;
    int ypos ;
    int theta ;
    int vel ;
} ;

```

where `xpos` and `ypos` give the (x, y) position of the robot relative to its starting point in units of mm. The `theata` member of the structure gives the orientation angle relative to its starting orientation in degrees. Finally, the vehicle velocity in mm/S is given by `vel`.

3.6 p3sonar

The `p3sonar` component is similar to the `p3loc` component in that it translates the P3 specific sensor data into a more generic form. It monitors the `P3_STATE` postings and presents a generic set of SONAR readings for other components to use.

It posts a `SONAR_STATE` message to the blackboard which is simply an array:

```

struct sonar_state {
    unsigned short srange[P3_MAX_SONAR] ;
} ;

```

of range measurements in units of mm.

3.7 trivob

This is a trivial obstacle handler. It is only a place-holder for the more general obstacle handler(s) to come. It watches the `SONAR_STATE` postings looking for an object directly in its path. Assuming that SONAR sensors 3 and 4 are pointed forward, if a reading of less than 500mm is take from either of them, we instruct the robot to stop. We assume that the navigator does not direct us toward a fixed obstacle, and thus anything we encounter is transient. Once the obstacle is gone, we proceed.

This component posts a `OB_STATE` message which simply has a go/no-go structure member:

```
struct ob_state {
    int op ;
} ;
```

where `ob` can take on the values `OB_STOP` and `OB_GO` with the obvious interpretations.

4 Writing a Component

In this section, we discuss how to go about writing a component to work in the system. We are taking the arbitrator as an example because it uses some features we don't see in other components, but at the same time is simple enough to describe in a reasonable amount of space. We start with the header file `arb.h`.

```
1  #ifdef CPLUSPLUS
2  extern "C" {
3  #endif
4
5  void *arb_init( void *dummy ) ;
6
7  #define ARB_CMD 0x00060001
8
9  #define ARB_STOP NAV_STOP
10 #define ARB_MOVE NAV_MOVE
11
12 struct arb_cmd {
13     int op ;
14     long vel ;
15     long radius ;
16 } ;
17
18 #ifdef CPLUSPLUS
19 }
20 #endif
```

Lines 1–3 and 18–20 are boilerplate just in case someone wants to link C++ code to this. None of the existing code is written using C++isms; it’s all straight C. However, just in case anyone wants to write something in C++, we can make provision for it as long as everyone agrees that external interfaces are C without the C++ name munging. Line 5 declares the only externally visible function for this component. Normally, the only function that is used by any other component is the main POSIX thread function. The return type and argument list is defined by the POSIX thread design. Every component should have such a function. As in this case, for a component called foo, the function should be called `foo_init()`. (More on naming conventions below.) In line 7, we define the symbolic name of the only data item we post to the blackboard. Each component is assigned a value to use for the upper 16 bits (in this case, 6) and the component designer may use the lower 16 bits anyway that’s appropriate. Lines 9 and 10 define symbolic values for one of the members of the structure posted to the blackboard. In this case, they are dependent on the navigator component. Finally, in lines 12–16, we define the actual data structure which is posted to the blackboard. Notice that all the definitions given in this header file relate to the items posted by this component and not to items posted by other components and used by this one.

Now we turn to the source in the file `arb.c`. In this example, this is the only source file for the component. For simple components like this one, this will be typical. For more complex components, there may well be multiple source files. When preparing multiples source files that export functions to each other, be careful about selecting names that are not likely to collide with those used by other components.

```

1  #include <stdio.h>
2  #include <car_agv.h>
3
4  void *arb_init( void *dummy )
5  {
6      struct arb_cmd cmd[2] ;
7      struct nav_cmd *p, last_nav ;
8      struct ob_state *q ;
9      long labels[] = { OB_STATE, NAV_CMD } ;
10     struct timeval *stamps[] = { NULL, NULL } ;
11     struct timeval stamp0, stamp1 ;
12     long l ;
13     int i, cur_obstate, which ;
14
15     cur_obstate = OB_GO ;
16     last_nav.op = NAV_MOVE ;
17     which = 1 ;
18     while( 1 ) {
19         l = bb_wait( labels, stamps, 2 ) ;
20         if( l < 0 ) {

```

```

21         fprintf( stderr, "bb_wait botched\n" ) ;
22         sleep( 1 ) ;
23         continue ;
24     }
25     switch( 1 ) {
26     case OB_STATE:
27         q = bb_copy( OB_STATE, NULL, &stamp0 ) ;
28         if( q ) {
29             stamps[0] = &stamp0 ;
30             cur_obstate = q->op ;
31             free( q ) ;
32         }
33         break ;
34     case NAV_CMD:
35         p = bb_copy( NAV_CMD, NULL, &stamp1 ) ;
36         if( p ) {
37             stamps[1] = &stamp1 ;
38             last_nav.op = p->op ;
39             last_nav.vel = p->vel ;
40             last_nav.radius = p->radius ;
41             free( p ) ;
42         }
43         break ;
44     }
45     which = 1 - which ;
46     if( cur_obstate == OB_STOP )
47         cmd[which].op = OB_STOP ;
48     else {
49         cmd[which].op = last_nav.op ;
50         cmd[which].vel = last_nav.vel ;
51         cmd[which].radius = last_nav.radius ;
52     }
53     bb_post( ARB_CMD, sizeof( struct arb_cmd ), &cmd[which] ) ;
54 }
55 }

```

In lines 1 and 2, we include a couple of header files. The first is the usual `stdio.h` which gets us the definition of `NULL` and which lets us use standard I/O for debugging. Line 2 is more relevant to the example. The file `car_agv.h` has all the definitions and includes for all the components in the system. Each component should include this and none of the other component header files directly. In the future, `car_agv.h` will be generated automatically by a configuration procedure, but for now it is hand generated. Line 4 defines the thread initialization function a la the declaration in `arb.h`. In line 6, we define an array of two arbitrator command structures. The reason for this is that we will be flip-flopping between the two as we load them and post them to the blackboard.

This idiom prevents race conditions while also keeping the blackboard from having to waste time on unnecessary copying. It's an easy idiom to follow, but we may still in the future add a blackboard call that obviates it. The remainder of the local variable declarations in lines 7–13 will be clear as we discuss their use.

Most components follow a basic pattern. They begin with some general initialization and then fall into an infinite loop. The loop is the very typical wait-process-output loop. Components generally spend most of their time waiting for either a posting to the blackboard or for an external input. Once they get that input, they do whatever processing is necessary and then either post results back to the blackboard or send control signals to some output device.

In this example, lines 15–17 are the general initialization. Effectively, we say that in the absence of any posting from either component, we assume we are to move. The `which` variable controls our toggling between the two command buffers. It switches between the values 0 and 1. It doesn't really matter which we use first, but since we complement it before using it, we initialize it to 1 so that the first one we use is 0. Line 19 is the wait for blackboard postings. We wait on two possible postings and will be awakened when either is posted. We check for a possible error in lines 20–24. The only errors that can occur in the current version of the blackboard are a failure to allocate memory and an “impossible” internal error where we are awakened by the addition of something we're waiting for but that's not there when we go to look it up. In most cases, it would be safe to ignore errors from `bb_wait()`.

Lines 25–44 are where we process the two posting we care about. The normal return value from `bb_wait()` tells us which one got posted, so we `switch` on it and process accordingly. In both cases, we use `bb_copy()` to retrieve the data which was posted. Most of the time, that's the better call to use. The `bb_read()` variant should only be used for large data structures (such as images) where the time to make a copy could be significant. In those cases, the components involved should implement their own locking mechanism to prevent data corruption. Also notice that in lines 28 and 36, we test to make sure that we actually got a data item back. In principle, this is another case of testing for an “impossible” error, but it doesn't take much code and it feels good. Next, both calls to `bb_copy()` get the timestamp of the posting and then store it in an array of last read timestamps. This is necessary if we want to receive only new postings rather than continually read the same ones over and over again, which is usually what we want. In both cases here, the real processing is just recording what we got for later use. Finally, we free the memory returned by `bb_copy()`. The client component is responsible for freeing memory allocated by `bb_copy()`, but **not** from `bb_read()`. Our use here is a bit sloppy since `bb_copy()` allocates memory and makes a copy, then we copy the data out of the copy and then free it. That will probably change in the future.

In lines 45–53, we then act on this new input. Line 45 is where we toggle the command buffer. We don't want to fiddle with the one currently on the blackboard in case someone makes a query while we're doing our thing. So we switch to the other one. The `if` statement in line 46 is where the arbitrator actually arbitrates. The obstacle handler has veto power over the navigator.

The two branches of the `if` set up a command structure which either says stop or says go according to the obstacle handler's and the navigator's most recent postings. Finally, in line 53, we post our results to the blackboard, assuming that there's a component listening which will turn that structure into whatever action is necessary to control the robot.

5 Naming and Numbering Conventions

This example also illustrates the proper method of choosing externally visible names. If a component is named *foo*, then any external symbols and structure tags should be prefixed with *foo_*. Likewise, any defined names in the component's header file should start with *FOO_*. Finally, the sub-directory where the source for the component is found should also be called *foo*.

We also have a convention regarding the assignment of blackboard labels. Each component is responsible for assigning the labels for data items it posts. Labels are divided into two 16-bit parts with the upper 16 bits assigned to the component and the lower 16 bit to be chosen by the component designer. The upper 16 bits are themselves divided into two 8-bit fields. The most significant 8 bits are assigned to the sub-groups within the center as follows:

Function	Lead	Number
Infrastructure/Hardware/Prototyping	Stuart	0
Localization	Stuart	1
Obstacle Handling	McCauley	2
Vision	Kahn	3
Cognitive Robotics	Franklin	4

Even though we have listed function areas in this table, the number assignment is really to the group lead. So for example, the trivial obstacle handler in the current prototype code uses a group ID of 0 rather than 2 as it would if it had come from the obstacle handling group. The next group of 8 bits are assigned to a component by the sub-group lead. The current assignments for the upper 16 bits are:

Component	Label
p3	1
nav	2
p3loc	3
p3sonar	4
trivob	5
arb	6

The project wiki page will maintain a list of the currently assigned numbers. When starting a new component, get a number assigned as soon as possible in order to minimize the chances of two components having the same numbers even temporarily.

6 Conclusion

This document describes the current state of AGV software as developed by the Center for Advanced Robotics. It is intended to be a living document, updated periodically as new components are added and major revisions are made to existing components. We cannot, however, guarantee that the document will never be out of date. The source code should always be considered more authoritative than this document.